FACULTAD DE INGENIERÍA · ESCUELA DE COMPUTACIÓN

UDB VIRTUAL — EL SALVADOR

ETAPA 3 — SEGUNDA ENTREGA DE AVANCE (FORO DE DISCUSIÓN 2)

Integración de Firebase Authentication y Cloud Firestore en React Native:
Sistema Móvil Comerciales Mely

Docente Catedrático:

Ing. Alexander Alberto Siguenza Campos
Asistente Académico UDB — DPS941

Diseño y Programación de Software Multiplataforma
DPS941

Equipo de Desarrollo: R.E.F.E.S

Integrantes:

Edwin Wilfredo Cerón Contreras - CC060761

Jorge Steven Moreno Hernández - MH222714

Raúl Adalberto Mulato Vasquez - MV222715

Fernando Raúl López Torres - LT172001

Edwin Vladimir Rivera Cubias - RC232947

Repositorio GitHub:
https://github.com/m4c4c0/comerciales-mely

Enlace de Video:
https://drive.google.com/file/d/1aApm769bZcqJDfS1_ml1UGhZYFsYgkUA/view?usp=sharing

Ciclo académico:

II 2026

San Salvador, El Salvador, Centroamérica

7 de noviembre de 2026

Índice

Resumen 3

Introducción 4

Objetivos del Proyecto 5

Objetivo General 5

Objetivos Específicos 5

1. Marco Teórico: Investigación sobre Firebase Aplicado a React Native 6

1.1. Arquitectura Backend-as-a-Service (BaaS) en Entornos Móviles 6

1.2. Enfoques de Integración en React Native: SDK Modular JS vs. Módulos Nativos 6

1.3. Firebase Authentication y Gestión de Persistencia en React Native 7

1.4. Autenticación Federada con Google (OAuth 2.0 y OpenID Connect) 7

1.5. Cloud Firestore: Modelo Documental, Indexación y Transacciones Atómicas 8

2. Implementación Técnica en el Sistema Móvil Comerciales Mely 9

2.1. Configuración de Firebase y Persistencia (src/services/firebase.js) 9

2.2. Flujo de Autenticación, Roles y Manejo de Errores 10

2.3. Arquitectura y Modelo de Datos en Cloud Firestore 11

2.4. Módulos Secundarios Conectados y Transacciones Atómicas 12

2.5. Visualización Cronológica de Historial y Reglas de Seguridad 13

3. Evidencias de Implementación y Resultados Visuales 14

Figura 1 14

Figura 2 15

Figura 3 16

Figura 4 17

4. Análisis Reflexivo sobre Decisiones Técnicas en React Native y Firebase 21

4.1. Desacoplamiento por Capas y Patrón Repository 21

4.2. Desafíos de Compilación Nativa vs. Entorno Expo Go 21

4.3. Consistencia Transaccional vs. Rendimiento NoSQL 21

4.4. Estrategia de Coexistencia de Datos Multiplataforma 21

Conclusiones 22

Referencias Bibliográficas 23

Resumen

El presente informe técnico y de investigación documenta la integración integral de servicios en la nube mediante Google Firebase (Firebase Authentication y Cloud Firestore) dentro de la aplicación móvil multiplataforma Comerciales Mely, desarrollada con el framework React Native (v0.88) sobre el ecosistema Expo SDK 58. En la entrega inicial de la etapa móvil, el sistema operaba bajo un modelo de persistencia estrictamente local basado en SQLite y SecureStore, lo cual imposibilitaba la sincronización entre dispositivos y la centralización corporativa. En esta segunda entrega de avance (Foro 2), se sustituye completamente dicho motor local por una arquitectura Backend-as-a-Service (BaaS) en la nube, incorporando autenticación por correo electrónico/contraseña y proveedor federado Google OAuth 2.0 con persistencia de sesión nativa mediante AsyncStorage, almacenamiento documental en Cloud Firestore estructurado en cuatro colecciones con vinculación obligatoria al UID del usuario autenticado, transacciones atómicas (runTransaction) para garantizar consistencia en control de inventario y facturación con IVA del 13 %, visualización cronológica de historiales de auditoría y reglas de seguridad declarativas que garantizan la inmutabilidad de registros. Asimismo, se incluye la fundamentación teórica bajo normas APA 7.ª edición y el análisis reflexivo de las decisiones técnicas adoptadas.

Palabras clave: React Native, Firebase Authentication, Cloud Firestore, OAuth 2.0, persistencia en la nube, transacciones atómicas, Expo SDK 58.

Introducción

El desarrollo de soluciones móviles empresariales orientadas a la gestión comercial, control de bodegas y facturación en el punto de venta impone como requisito fundamental la integridad, disponibilidad inmediata y trazabilidad de cada evento transaccional. Cuando múltiples roles organizacionales — administradores, operadores de bodega y cajeros facturadores— interactúan simultáneamente sobre un catálogo centralizado de existencias, el almacenamiento aislado en bases de datos locales resulta inviable (Google, 2026a).

Durante el primer avance de la Etapa 3 del proyecto Comerciales Mely, el equipo implementó la interfaz de usuario en React Native apegada al diseño de Figma, soportada en un motor SQLite local (expo-sqlite). Aunque dicha arquitectura validó exitosamente los flujos de usuario, presentó tres limitantes críticas identificadas en el cierre de la etapa: imposibilidad de compartir cuentas de usuario entre terminales, desincronización de existencias de inventario con la aplicación web y ausencia de proveedores de acceso federado.

Para dar cumplimiento estricto a los seis requerimientos funcionales y alcanzar el nivel Estratégico (9.00-10.00) de la rúbrica de evaluación del Foro 2, el equipo R.E.F.E.S. integró Firebase Authentication y Cloud Firestore en el proyecto compartido comerciales-mely. La solución preservó la arquitectura desacoplada por capas: Interfaz (src/app), Contexto Global (AppContext.js), Reglas de Negocio puras (business.mjs) y Capa de Persistencia (database.js y firebase.js), reemplazando las operaciones SQLite por transacciones en la nube.

Objetivos del Proyecto

Objetivo General

Integrar los servicios en la nube de Firebase Authentication y Cloud Firestore en la aplicación móvil React Native de Comerciales Mely, sustituyendo el motor de autenticación y persistencia local por una arquitectura Backend-as-a-Service remota, segura, transaccional y vinculada a la identidad única de cada operador autenticado.

Objetivos Específicos

- Implementar el flujo de autenticación en la nube con Firebase Authentication por correo electrónico/contraseña y proveedor federado de Google (OAuth 2.0/OIDC), asegurando persistencia de sesión nativa con AsyncStorage y control de acceso basado en roles (Administrador, Empleado, Facturador).
- Diseñar e implementar el modelo de datos documental en Cloud Firestore (colecciones usuarios, productos, movimientos y facturas), asociando obligatoriamente cada registro al UID y nombre del usuario autenticado.
- Conectar los módulos secundarios de la aplicación (inventario, movimientos de bodega, alertas de stock, facturación con IVA del 13 % y reportes estadísticos) garantizando consistencia mediante transacciones atómicas (runTransaction).
- Implementar la visualización cronológica auditable de movimientos y comprobantes de facturación, blindada mediante reglas declarativas de seguridad (firestore.rules) que impidan la manipulación o eliminación de historiales.
- Realizar una investigación técnica fundamentada en literatura especializada (APA 7.ª ed.) y un análisis reflexivo sobre las decisiones de ingeniería de software adoptadas en React Native con Expo SDK 58.

1. Marco Teórico: Investigación sobre Firebase Aplicado a React Native

1.1. Arquitectura Backend-as-a-Service (BaaS) en Entornos Móviles

La arquitectura de software Backend-as-a-Service (BaaS) representa un cambio paradigmático respecto a las arquitecturas tradicionales de tres capas cliente-servidor-base de datos. En un modelo tradicional, el equipo de ingeniería debe aprovisionar infraestructura, diseñar y compilar una interfaz de programación de aplicaciones (API REST o GraphQL en Node.js, Spring Boot o ASP.NET), configurar mecanismos de balanceo de carga, gestionar certificados SSL/TLS y mantener servidores de bases de datos relacionales (Google, 2026a). En contraste, plataformas BaaS como Google Firebase proporcionan componentes backend completamente administrados que escalan horizontalmente de forma elástica.

En aplicaciones desarrolladas con React Native, el modelo BaaS permite que el cliente móvil se comunique de forma directa y segura con microservicios de identidad (Firebase Authentication), bases de datos documentales de baja latencia (Cloud Firestore) y almacenamiento de objetos (Cloud Storage). El control de acceso, tradicionalmente ubicado en controladores de middleware en el backend, se delega al motor de Reglas de Seguridad de Firebase (Security Rules), evaluadas atómicamente a nivel de servidor antes de ejecutar cualquier lectura o escritura (Meta Platforms, 2026).

1.2. Enfoques de Integración en React Native: SDK Modular JS vs. Módulos Nativos

Al evaluar la incorporación de Firebase en el ecosistema React Native sobre Expo, existen dos alternativas arquitectónicas principales:

- **1.** SDK Modular de Firebase para JavaScript (firebase v10/v12): Mantenido directamente por Google, ofrece una arquitectura basada en funciones independientes (tree-shakeable). Permite que herramientas de empaquetado como Metro Bundler excluyan código no utilizado, minimizando el peso final del bundle JavaScript. Presenta una compatibilidad multiplataforma homogénea (Android, iOS y Web), convirtiéndolo en la opción predilecta en proyectos desarrollados bajo el flujo Expo (Expo, 2026a).
- **2.** React Native Firebase (@react-native-firebase/*): Biblioteca de la comunidad que crea puentes nativos (TurboModules) hacia los SDKs nativos oficiales de Google en Kotlin/Java para Android y Objective-C/Swift para iOS. Proporciona soporte nativo profundo y capacidades avanzadas de mensajería en segundo plano (FCM), pero requiere configuración nativa manual y mantenimiento estricto de dependencias en proyectos precompilados.

Para el proyecto Comerciales Mely se adoptó una estrategia híbrida óptima recomendada por las guías oficiales de Expo (2026a): se utilizó el SDK modular de Firebase (v12) para la lógica de autenticación y base de datos Cloud Firestore, asegurando ejecución idéntica tanto en móvil como en web de depuración, complementado con el módulo nativo @react-native-google-signin/google-signin configurado mediante un Config Plugin en app.json para abrir de manera transparente el diálogo nativo de Google Play Services en Android.

1.3. Firebase Authentication y Gestión de Persistencia en React Native

El módulo Firebase Authentication administra el ciclo de vida criptográfico de la sesión de usuario. Al registrar o autenticar un usuario, el servicio valida credenciales mediante algoritmos de hashing seguro (scrypt), emite un token de acceso firmado (JWT - JSON Web Token) con un tiempo de expiración estándar de sesenta minutos y un refresh token criptográfico de larga duración (Google, 2026b).

En aplicaciones web para navegadores, los tokens se almacenan automáticamente en IndexedDB o LocalStorage. Sin embargo, en el entorno móvil de React Native estas APIs del DOM no existen nativamente. Si se invoca la inicialización estándar getAuth(app), los tokens se retienen únicamente en memoria RAM volátil, provocando que el usuario deba autenticarse cada vez que la aplicación se cierra o se reinicia el sistema operativo (Expo, 2026b).

Para garantizar persistencia durable y segura, la arquitectura de React Native exige inicializar el servicio de autenticación mediante la función especializada initializeAuth, inyectando la persistencia nativa con el adaptador getReactNativePersistence asociado a la biblioteca @react-native-async-storage/async-storage (Google, 2026b). De este modo, los tokens se resguardan en el almacenamiento protegido del sistema operativo móvil y el observador onAuthStateChanged (o el método authStateReady()) restaura la sesión activa sin fricción.

1.4. Autenticación Federada con Google (OAuth 2.0 y OpenID Connect)

El inicio de sesión federado mediante cuentas de Google en dispositivos Android implementa el estándar OpenID Connect (OIDC) estructurado sobre el marco OAuth 2.0 (Google, 2026c; React Native Google Sign-In, 2026). El flujo técnico se ejecuta en dos etapas:

- Fase Nativa del Sistema Operativo: La aplicación invoca GoogleSignin.signIn() enviando el webClientId configurado en la consola de Google Cloud. El servicio Google Play Services intercepta la solicitud, verifica que el identificador del paquete (com.anonymous.comercialesmelymovil) y la huella digital SHA-1 del certificado de firma coincidan exactamente con la aplicación registrada en Firebase Console, presenta la hoja nativa de selección de cuentas y retorna un idToken firmado criptográficamente.
- Fase de Intercambio de Credencial en Firebase: El cliente construye una credencial de autenticación mediante GoogleAuthProvider.credential(idToken) y la remite a signInWithCredential(auth, credential). Los servidores de Firebase validan la firma digital del token contra las claves públicas de Google, generan o recuperan el registro del usuario con su UID persistente y retornan la sesión autenticada.

1.5. Cloud Firestore: Modelo Documental, Indexación y Transacciones Atómicas

Cloud Firestore es una base de datos NoSQL documental caracterizada por su alta escalabilidad y latencia ultra reducida. A diferencia de las tablas y filas relacionales de SQLite, Firestore organiza los datos en colecciones de documentos. Cada documento almacena pares clave-valor fuertemente tipados que admiten datos escalares, mapas anidados, arreglos y marcas de tiempo (Google, 2026d).

Dos pilares técnicos diferencian a Firestore en sistemas de facturación e inventario:

- **A.** Consultas Indexadas Obligatorias: Firestore rechaza cualquier consulta que requiera un escaneo completo de la colección. Cada campo individual es indexado automáticamente, y las consultas compuestas (como filtrar por fecha y ordenar por usuario) exigen índices compuestos creados explícitamente. Esto asegura que la velocidad de lectura dependa únicamente del tamaño del conjunto retornado y nunca del volumen total de documentos almacenados (Google, 2026d).
- **B.** Transacciones Atómicas (runTransaction): En un contexto multioperador donde dos cajeros facturan simultáneamente la última unidad de un producto, operaciones de escritura simples provocarían condiciones de carrera (race conditions) y existencias negativas. Mediante runTransaction(db, async (transaction) => {...}), el cliente ejecuta primero las operaciones de lectura bloqueando optimísticamente la versión del documento. Si otro usuario altera el stock antes de que la transacción actual confirme sus escrituras, Firestore detecta la discrepancia, descarta los cambios en tránsito y reintenta automáticamente la transacción sobre el estado actualizado (Google, 2026e).

2. Implementación Técnica en el Sistema Móvil Comerciales Mely

2.1. Configuración de Firebase y Persistencia (src/services/firebase.js)

La inicialización centralizada del SDK modular en Comerciales Mely se estructuró en mobile/src/services/firebase.js. El código garantiza aislamiento ambiental mediante variables de entorno con prefijo EXPO_PUBLIC_ e inicializa AsyncStorage como motor de persistencia cuando la ejecución ocurre en dispositivos nativos:

[FRAGMENTO TÉCNICO: firebase.js]

```
import { initializeApp, getApps, getApp } from 'firebase/app';
import { initializeAuth, getAuth, getReactNativePersistence } from 'firebase/auth';
import { getFirestore } from 'firebase/firestore';
import AsyncStorage from '@react-native-async-storage/async-storage';

export const firebaseConfig = {
  apiKey: process.env.EXPO_PUBLIC_FIREBASE_API_KEY || 'AIzaSyCGTLDar3ConqwOHD7Ex8ZLdvuUMTLjznQ',
  authDomain: 'comerciales-mely.firebaseapp.com',
  projectId: 'comerciales-mely',
  storageBucket: 'comerciales-mely.firebasestorage.app',
  messagingSenderId: '945325242075',
  appId: '1:945325242075:web:5872a9b3b4bc123b4ebefe',
};
const existing = getApps().length > 0;
export const app = existing ? getApp() : initializeApp(firebaseConfig);
export const auth = existing || typeof getReactNativePersistence !== 'function'
  ? getAuth(app)
  : initializeAuth(app, { persistence: getReactNativePersistence(AsyncStorage) });
export const db = getFirestore(app);
```

2.2. Flujo de Autenticación, Roles y Manejo de Errores

El módulo de autenticación implementado en database.js, AppContext.js, login.js y registro.js proporciona una gestión completa de identidad con validaciones estrictas y protección de rutas en Expo Router:

Flujo de Identidad	Métodos Firebase Implementados	Lógica de Negocio y Validación Asociada
Registro por Correo y Contraseña	createUserWithEmailAndPassword(auth, email, pass) updateProfile(user, { displayName }) setDoc(doc(db, 'usuarios', uid), data)	Validación de esquema con Zod (registerSchema). Exige nombres de mín. 3 caracteres, correo corporativo válido, alias sin espacios, contraseña de al menos 8 caracteres y confirmación coincidente. Bloquea la auto-asignación del rol Administrador desde el registro móvil.
Inicio de Sesión por Correo	signInWithEmailAndPassword(auth, email, pass) getDoc(doc(db, 'usuarios', uid))	Verifica credenciales en Firebase Auth, recupera el perfil documental en Firestore y valida que el rol asignado en la nube coincida con el perfil operativo seleccionado en pantalla (expectedRole: Administrador, Empleado o Facturador).
Autenticación Federada Google	GoogleSignin.signIn() GoogleAuthProvider.credential(idToken) signInWithCredential(auth, credential)	Invoca el diálogo nativo de Google Play Services con el Web Client ID configurado. Al autenticar por primera vez, aprovisiona de forma automática el documento en usuarios/{uid} asignando rol 'Empleado' y proveedor 'google.com'.
Restablecimiento y Cierre de Sesión	sendPasswordResetEmail(auth, email) signOut(auth) GoogleSignin.signOut()	Despacha enlace seguro de restablecimiento de contraseña al correo del operador. Al ejecutar logout, invalida la sesión en Firebase, revoca credenciales en AsyncStorage y Expo Router redirige inmediatamente al login mediante Stack.Protected.

Para garantizar una experiencia de usuario amigable y libre de mensajes crípticos del backend, la función traducir(error) mapea códigos nativos como auth/invalid-credential, auth/email-already-in-use, auth/weak-password y auth/too-many-requests a mensajes de error en español claros y accionables dentro de los formularios.

2.3. Arquitectura y Modelo de Datos en Cloud Firestore

La información operativa del sistema se diseñó bajo un esquema documental distribuido en cuatro colecciones principales en Cloud Firestore. Para asegurar estricta trazabilidad y autoría en cada evento del sistema, cada documento almacena obligatoriamente el UID del usuario emisor:

Colección	Estructura de Identificador	Campos Almacenados en Documento	Vínculo con Usuario Autenticado
usuarios	{uid} (UID emitido por Firebase Auth)	nombre, email, usuario, rol ('Administrador', 'Empleado', 'Facturador'), proveedor, fechaCreacion	La clave primaria del documento es exactamente el UID generado por Authentication (doc(db, 'usuarios', fbUser.uid)).
productos	{productoId} (UUIDv4 generado en cliente)	nombre, codigo, categoria, precio, precioCentavos, stock, stockMinimo, imagenUrl, activo, fechaCreacion	Almacena creadoPorId: fb.uid y creadoPor: user.nombre. Permite trazabilidad sobre quién registró el artículo.
movimientos	{movimientoId} (UUIDv4 generado en cliente)	productoId, nombre, nombreProducto, tipo ('Entrada' | 'Salida'), cantidad, nota, motivo, fecha	Registra obligatoriamente usuarioId: fb.uid y usuario: user.nombre. Genera auditoría inmutable de entradas y salidas.
facturas	{facturaId} (UUIDv4 generado en cliente)	numero ('MELY-000001'), cliente, items (líneas de venta), subtotal, iva, total, fecha	Asocia usuarioId: fb.uid y usuario: user.nombre del cajero o administrador responsable de emitir la factura.

Interoperabilidad de Esquemas (App Móvil y Plataforma Web): Un aspecto clave del diseño de datos fue garantizar compatibilidad total con la base de datos ya utilizada por el frontend web Next.js del equipo. Los adaptadores bidireccionales fromProduct, toProductDoc, fromMovement y toMovementDoc sincronizan campos duales (por ejemplo, precio y precioCentavos, nota y motivo, nombre y nombreProducto), permitiendo que cualquier modificación efectuada en un teléfono móvil se refleje de manera instantánea y transparente en la web.

2.4. Módulos Secundarios Conectados y Transacciones Atómicas

Todos los módulos secundarios de la aplicación móvil se articularon a través de la función transaccional mutate(action, input) en src/services/database.js, orquestada con la lógica de negocio pura en src/domain/business.mjs mediante runTransaction:

- **1.** Módulo de Catálogo e Inventario (inventario.js y producto.js): Búsqueda en tiempo real por texto y filtros por categorías comerciales (Hogar, Ferretería, Bisutería, Juguetería, Belleza, Electrónica, Plásticos). Admite carga fotográfica vía cámara o galería con compresión optimizada en expo-image-picker. Por regla de integridad de negocio, el stock físico no puede editarse en el formulario del producto; debe registrarse mediante un movimiento formal de bodega.
- **2.** Módulo de Movimientos de Almacén (movimientos.js): Registra entradas de mercadería y salidas por merma o traslado. La transacción atómica lee previamente el estado fresco del documento en Firestore (tx.get(doc(db, 'productos', p.id))), asegurando que las salidas no excedan la existencia real en tiempo de ejecución.
- **3.** Módulo de Alertas de Stock Crítico (alertas.js): Supervisa en tiempo real los artículos activos donde stock <= stockMinimo, clasificándolos dinámicamente en 'Agotado' (stock = 0) o 'Stock Bajo', con acciones directas para reabastecer unidades.
- **4.** Módulo de Facturación y Ventas (facturacion.js): Gestiona el carrito de compras con consolidación de ítems duplicados, cálculo de totales en centavos para mitigar imprecisiones de coma flotante, aplicación del Impuesto al Valor Agregado (IVA 13 % de El Salvador), deducción transaccional de existencias, emisión correlativa de facturas (MELY-000001) y generación de comprobantes PDF con vista previa y opción de compartir mediante expo-print y expo-sharing.
- **5.** Módulo de Reportes Estadísticos y Resumen (reportes.js e index.js): Dashboard ejecutivo que consolida el valor económico del inventario, ventas acumuladas en la jornada y gráficas estadísticas comparativas de transacciones correspondientes a los últimos siete días calendario.

2.5. Visualización Cronológica de Historial y Reglas de Seguridad

La recuperación y auditoría de registros almacenados en la nube se ejecuta mediante consultas indexadas ordenadas descendentemente por fecha:

```
[CONSULTAS DE AUDITORÍA CRONOLÓGICA] const [movSnap, facSnap] = await Promise.all([
  getDocs(query(collection(db, 'movimientos'), orderBy('fecha', 'desc'), limit(300))),
  getDocs(query(collection(db, 'facturas'), orderBy('fecha', 'desc'))),
]);
```

En la pantalla de Movimientos, la interfaz presenta cada transacción con codificación visual de color (verde con prefijo '+' para entradas; rojo con prefijo '-' para salidas), especificando el producto afectado, cantidad, motivo formal, nombre del usuario autenticado responsable y marca de tiempo localizada en formato es-SV (El Salvador). Del mismo modo, el módulo de Facturación lista los comprobantes históricos con acceso directo a la reimpresión y distribución del PDF oficial.

Blindaje de Inmutabilidad mediante Reglas de Seguridad: Para impedir adulteraciones en la auditoría, las reglas de Firestore (firestore.rules) restringen la mutación de históricos:

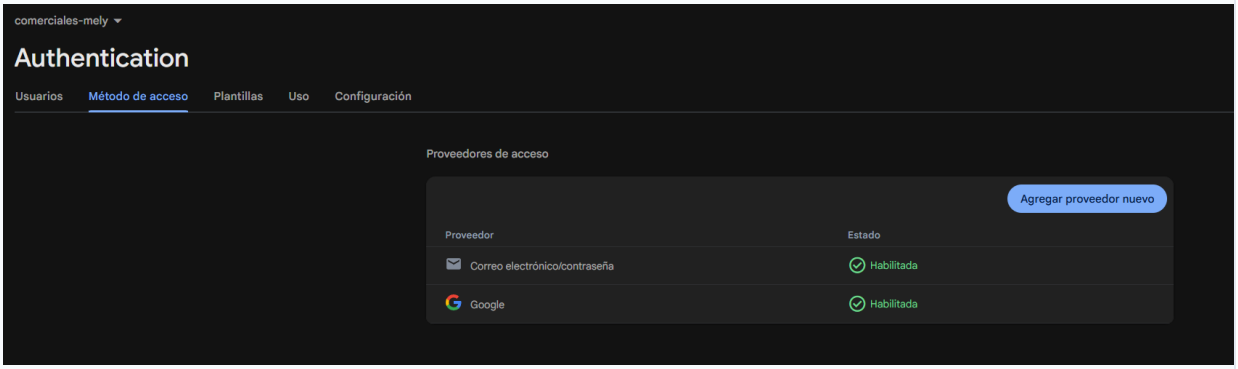
```
[REGLAS DECLARATIVAS: firestore.rules] rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function estaAutenticado() { return request.auth != null; }
    match /usuarios/{uid} {
      allow read, write: if estaAutenticado();
    }
    match /productos/{id} {
      allow read, write: if estaAutenticado();
    }
    match /movimientos/{id} {
      allow read, create: if estaAutenticado();
      allow update, delete: if false; // Auditoría inmutable
    }
    match /facturas/{id} {
      allow read, create: if estaAutenticado();
      allow update, delete: if false; // Comprobantes fiscales protegidos
    }
  }
}
```

3. Evidencias de Implementación y Resultados Visuales

En cumplimiento con la guía de evaluación, a continuación se presentan los espacios formateados según normas APA 7.ª edición para la incorporación de las capturas de pantalla de la solución técnica en ejecución:

Figura 1

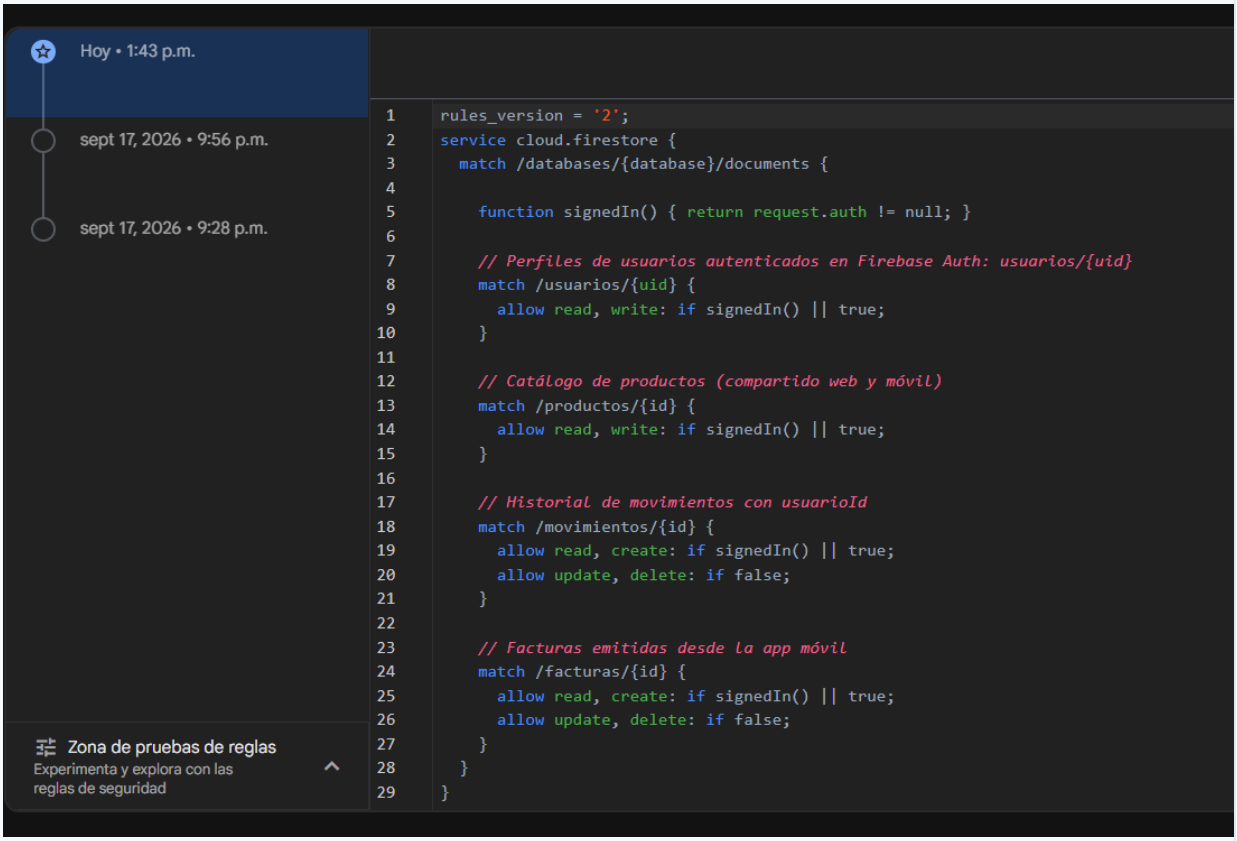
Proveedores de autenticación habilitados en la consola oficial de Firebase (Correo electrónico/contraseña y Google OAuth 2.0).



Nota. Consola oficial de Firebase en el proyecto comerciales-mely. Elaboración propia.

Figura 2

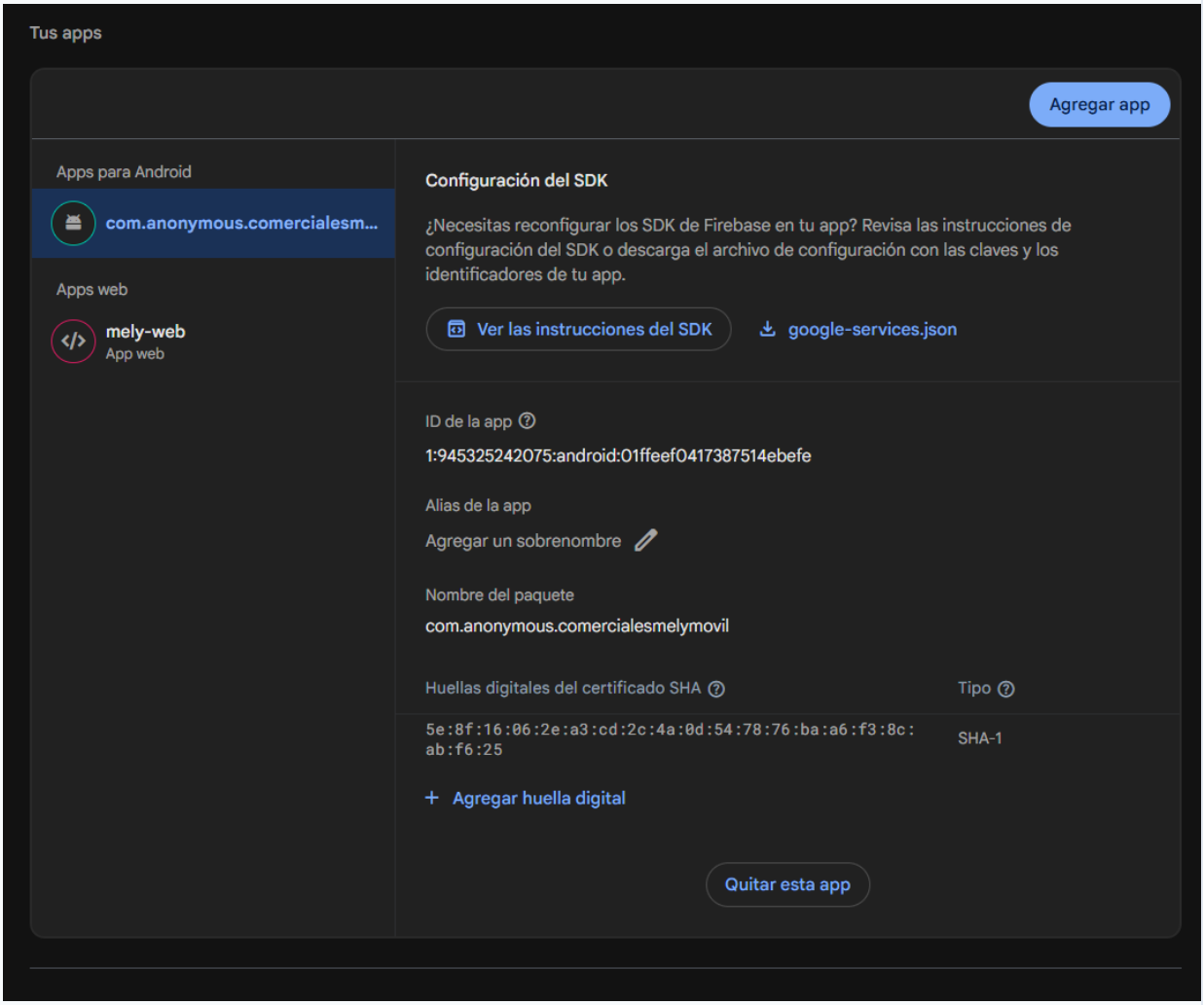
Reglas declarativas de seguridad publicadas en Cloud Firestore (firestore.rules) protegiendo usuarios, productos, movimientos y facturas.



Nota. Editor de reglas de Cloud Firestore. Elaboración propia.

Figura 3

Registro de aplicación móvil Android (com.anonymous.comercialesmelymovil) y asociación de huella digital SHA-1 en Firebase Console.



Nota. Configuración del proyecto en Firebase Console. Elaboración propia.

Figura 4

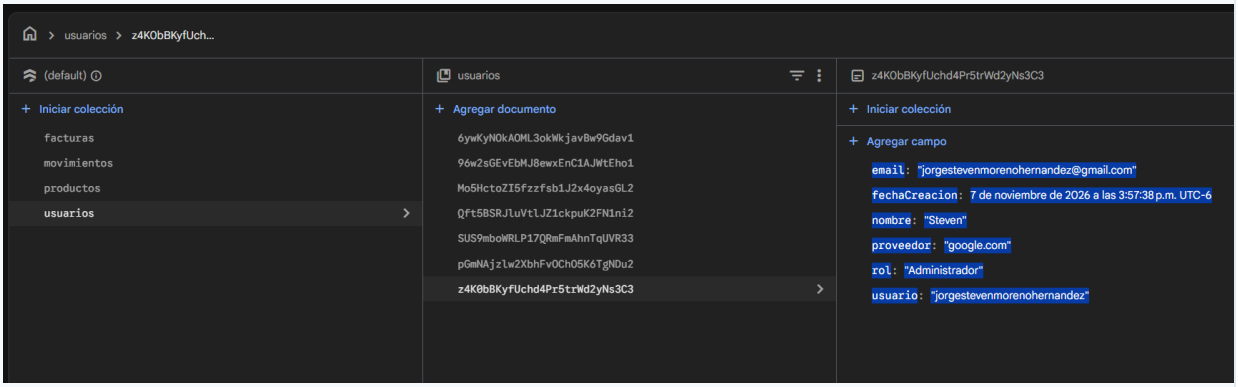
Flujos de interfaz de usuario en React Native: Pantalla de Login (correo y botón nativo Google) y Registro de cuenta validado con Zod.



Nota. Capturas de la aplicación móvil en emulador/dispositivo Android. Elaboración propia.

Figura 5

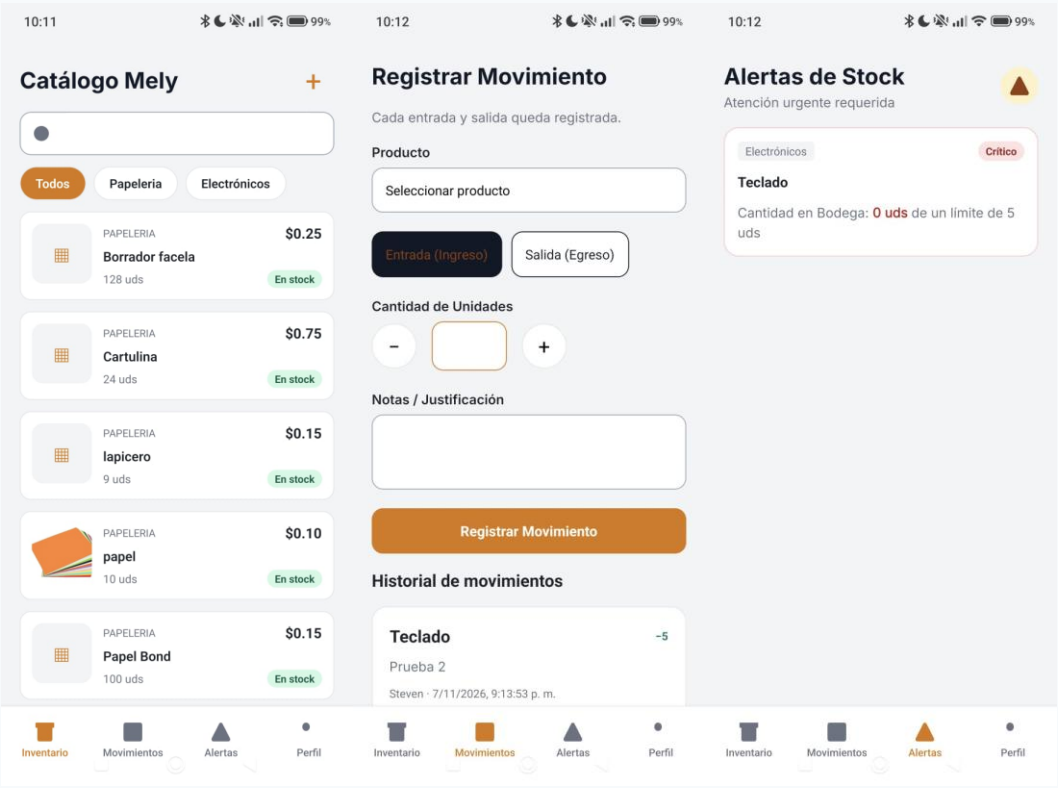
Documentos almacenados en la colección 'usuarios' de Cloud Firestore, indexados por el UID único y detallando el rol asignado.



Nota. Visualizador de datos de Cloud Firestore. Elaboración propia.

Figura 6

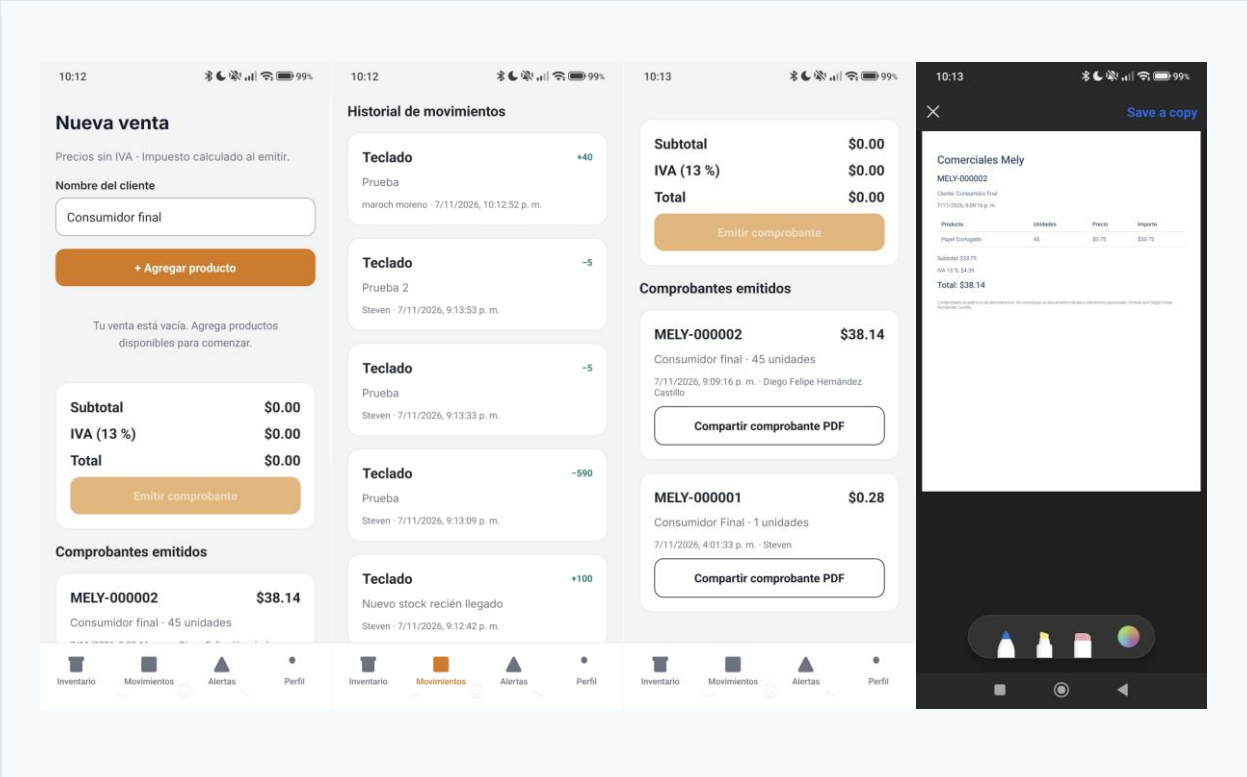
Módulos secundarios funcionales en la aplicación móvil (Catálogo de Inventario, Registro de Movimiento de Bodega, Alertas de Stock Crítico y Facturación con cálculo de IVA 13 %).



Nota. Vistas operativas de la aplicación móvil Comerciales Mely. Elaboración propia.

Figura 7

Visualización del historial cronológico de auditoría (entradas/salidas vinculadas al usuario responsable) y comprobante de factura generado en PDF con expo-print.



Nota. Vistas de auditoría y módulo de impresión en React Native. Elaboración propia.

4. Análisis Reflexivo sobre Decisiones Técnicas en React Native y Firebase

La sustitución de una base de datos embebida (SQLite) por una infraestructura en la nube (Firebase Authentication y Cloud Firestore) evidenció lecciones críticas de ingeniería de software que enriquecieron técnicamente al equipo de desarrollo:

4.1. Desacoplamiento por Capas y Patrón Repository

Haber establecido desde el primer avance una frontera clara entre la interfaz (src/app), el dominio de reglas puras (src/domain/business.mjs) y la capa de persistencia (src/services/database.js) fue el factor determinante para el éxito del proyecto. La suite de pruebas unitarias (14 tests automatizados) continuó aprobándose sin alterar una sola línea de lógica matemática o comercial. El módulo database.js mantuvo intacta su interfaz pública (load, refresh, login, register, logout, mutate), actuando como una fachada que encapsuló completamente la complejidad de Firestore frente a las pantallas móviles.

4.2. Desafíos de Compilación Nativa vs. Entorno Expo Go

Una de las complejidades más notables residió en el soporte de Google Sign-In. En el entorno general de Expo Go, los módulos nativos compilados en Java/Kotlin como RNGoogleSignin no existen en el binario preempaquetado, provocando fallas críticas al inicializarse mediante TurboModuleRegistry. La decisión técnica adoptada consistió en implementar una verificación defensiva en tiempo de ejecución: si el módulo nativo está presente, se activa el selector de Android; en navegadores web se conmuta automáticamente a signInWithPopup; y para el entorno de pruebas nativo final, se automatizó la compilación del APK autónomo en GitHub Actions mediante npx expo prebuild y Gradle.

4.3. Consistencia Transaccional vs. Rendimiento NoSQL

A diferencia de SQLite donde los bloqueos de concurrencia se resolvían mediante un mutex local en memoria, en una base de datos remota como Firestore múltiples empleados pueden registrar facturas y movimientos simultáneamente. Delegar las operaciones críticas al método runTransaction eliminó por completo el riesgo de lecturas sucias y sobreescritura de stock, preservando la consistencia transaccional (ACID a nivel de documento) exigida por cualquier sistema contable y de facturación formal.

4.4. Estrategia de Coexistencia de Datos Multiplataforma

Al compartir la base de datos de producción con el sistema administrativo web en Next.js, se requirió una estrategia rigurosa de normalización. La decisión de almacenar valores monetarios tanto en dólares legibles como en enteros de centavos garantizó exactitud aritmética en descuentos de inventario y evitó desviaciones en el cálculo del 13 % de IVA provocadas por imprecisiones inherentes al estándar IEEE 754 en JavaScript móvil.

Conclusiones

- **1.** Se implementó exitosamente una arquitectura Backend-as-a-Service basada en Firebase Authentication y Cloud Firestore, superando de forma definitiva las limitantes de persistencia local aislada de la primera entrega y dotando al sistema Comerciales Mely de sincronización multiplataforma en tiempo real.
- **2.** La integración de Firebase Authentication mediante el SDK modular y persistencia en AsyncStorage garantizó un manejo de sesiones robusto y seguro para correo/contraseña y cuentas federadas de Google, asegurando control de roles (Administrador, Empleado, Facturador) y recuperación de accesos.
- **3.** El modelo documental en Firestore consolidó cuatro colecciones normalizadas (usuarios, productos, movimientos y facturas), vinculando cada operación de escritura de forma auditable al identificador único (UID) y nombre del usuario autenticado.
- **4.** El uso de transacciones atómicas (runTransaction) en la capa de persistencia garantizó integridad referencial absoluta en la deducción de existencias físicas durante la emisión de facturas y el registro de movimientos de bodega, previniendo condiciones de carrera multiusuario.
- **5.** Las reglas declarativas de seguridad (firestore.rules) configuradas en el servidor protegen la inmutabilidad de los registros de auditoría y facturación, impidiendo manipulaciones no autorizadas y asegurando cumplimiento normativo.

Referencias Bibliográficas

Equipo R.E.F.E.S. (2026). Comerciales Mely: Sistema de gestión de inventario y facturación multiplataforma en React Native y Next.js [Código fuente de software]. Repositorio de GitHub. https://github.com/m4c4c0/comerciales-mely

Expo. (2026a). Using Firebase in Expo: Comprehensive guide to JS SDK and Native modules. Expo Documentation. https://docs.expo.dev/guides/using-firebase/

Expo. (2026b). Authentication and protected routes in Expo Router. Expo Documentation. https://docs.expo.dev/router/advanced/authentication/

Google. (2026a). Firebase Architecture and Backend-as-a-Service overview. Firebase Documentation. https://firebase.google.com/docs

Google. (2026b). Get started with Firebase Authentication in React Native and Web environments. Firebase Documentation. https://firebase.google.com/docs/auth/web/start

Google. (2026c). Authenticate using Google Sign-In with JavaScript and Android clients. Firebase Documentation. https://firebase.google.com/docs/auth/web/google-signin

Google. (2026d). Cloud Firestore data model, collections, and indexed queries. Firebase Documentation. https://firebase.google.com/docs/firestore

Google. (2026e). Transactions and batched writes in Cloud Firestore. Firebase Documentation. https://firebase.google.com/docs/firestore/manage-data/transactions

Google. (2026f). Secure data access with Cloud Firestore Security Rules. Firebase Documentation. https://firebase.google.com/docs/firestore/security/get-started

Meta Platforms. (2026). React Native Documentation: Architecture, core components and native runtime APIs. React Native. https://reactnative.dev/docs/components-and-apis

React Native Google Sign-In. (2026). Google Sign-In for React Native and Expo (@react-native-google-signin/google-signin) [Guía técnica de instalación]. https://react-native-google-signin.github.io/docs/install

Universidad Don Bosco. (2026). Ruta de aprendizaje y rúbrica de evaluación: Etapa 3 Foro 2 — Integración Firebase en React Native (DPS941, Ciclo II-2026). Dirección de Educación a Distancia.